ASSIGNMENT 1

Web Development Fundamentals
Frontend, Backend, Client-Server Architecture, Web Servers and Web Development Environment

1. Difference Between Frontend, Backend, and Full-Stack Development

Frontend development focuses on the part of a website or web application that users see and interact with. It commonly uses HTML for structure, CSS for styling, and JavaScript for behaviour. **Example:** the product page, navigation bar, buttons, and shopping cart interface on an e-commerce website.

Backend development focuses on server-side logic, authentication, business rules, APIs, and communication with databases. Common technologies include Java, Python, JavaScript/Node.js, and PHP. **Example:** when a customer places an order, the backend validates the order, calculates the total, stores it in the database, and returns a response.

Full-stack development involves working across frontend and backend, including database and application integration. **Example:** a full-stack developer may build the e-commerce interface, create the order API, implement authentication, and connect the application to a database.

2. Client-Server Model Diagram

The client (usually a browser) sends an HTTP/HTTPS request to a server. The server processes the request and sends back a response such as HTML, CSS, JavaScript, JSON, images, or other resources.

3. How a Browser Requests and Displays a Web Page

1. URL entry: The user enters a URL in the browser.

2. DNS lookup: The domain name is resolved to the server's IP address.

3. Connection: The browser establishes a network connection, normally using TCP and TLS for HTTPS.

4. HTTP request: The browser requests the required resource, usually the HTML document.

5. Server response: The server returns an HTTP response containing the requested resource and status code.

6. Resource loading: The browser parses HTML and discovers CSS, JavaScript, images, fonts, and other resources.

7. Rendering: The browser builds the DOM from HTML and CSSOM from CSS, calculates layout, paints pixels, and composites the page.

8. Interaction: JavaScript responds to user actions and can modify the page.

4. Tools Required for a Web Development Environment

Tool	Purpose
Code editor / IDE	Write, edit, organise, and debug HTML, CSS, and JavaScript. Example: Visual Studio Code.
Web browser	Run and test websites using a browser's rendering engine and developer tools.
Browser DevTools	Inspect HTML/CSS, debug JavaScript, inspect network requests, storage, and performance.
Git	Track source-code changes and collaborate with other developers.
GitHub / Git hosting	Store repositories online, collaborate, review code, and manage versions.
Node.js + npm	Run JavaScript outside the browser and install/manage development packages.
Terminal / Command Prompt	Run Git, Node.js, package-manager, build-tool, and development-server commands.

5. What is a Web Server?

A **web server** is software, and sometimes the computer running that software, that receives HTTP/HTTPS requests and provides web resources or dynamically generated responses to clients. It can serve HTML, CSS, JavaScript, images, videos, and other resources.

Common examples: Apache HTTP Server, Nginx, Microsoft IIS, and Caddy. Node.js can also provide HTTP server functionality, often through frameworks such as Express.

6. Roles in a Web Development Project

Role	Main Responsibilities
Frontend Developer	Builds the user interface; writes HTML, CSS, and JavaScript; makes pages responsive and accessible; inte
Backend Developer	Builds server-side logic, authentication/authorisation, APIs, validation, business rules, and database integra
Database Administrator (DBA)	Designs and maintains databases; manages users and permissions, backups, recovery, performance, avai

7. Installing and Configuring VS Code for HTML, CSS, and JavaScript

Step 1: Download and install Visual Studio Code.

Step 2: Open VS Code and create a folder such as *WebDevelopment*.

Step 3: Create *index.html*, *style.css*, and *script.js*.

Step 4: Link CSS and JavaScript files from the HTML file.

Step 5: Install useful extensions such as Live Server and Prettier.

Step 6: Run the HTML page using Live Server or a browser and use DevTools for debugging.

> **Screenshot requirement:** After completing the setup, take a screenshot showing VS Code with the HTML/CSS/JavaScript project open and insert that screenshot before submission. A personal installation screenshot cannot be fabricated for you.

8. Static vs Dynamic Websites

Feature	Static Website	Dynamic Website
Content	Usually fixed files served as they are.	Content can be generated or changed based on users, data, or req

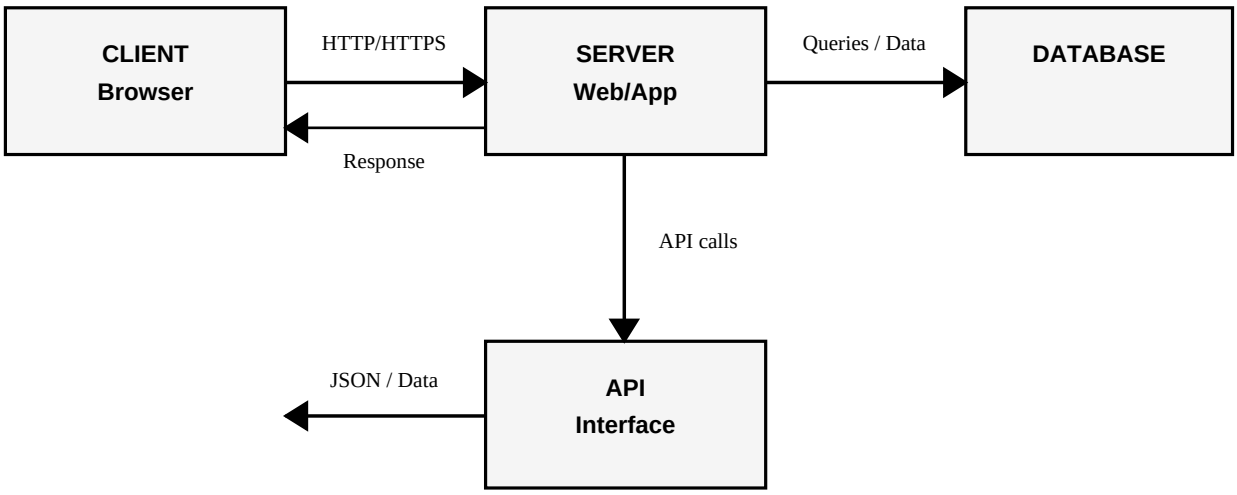
Feature	Static Website	Dynamic Website
Backend	May not require application-side processing.	Usually involves server-side application logic and/or APIs.
Database	Not necessarily required.	Often uses a database.
Example	Simple portfolio or documentation site made from HTML/CSS.	E-commerce site where products, carts, accounts, and orders are h

9. Five Web Browsers and Their Rendering Engines

Browser	Rendering Engine	Notes
Google Chrome	Blink	Chromium-based; Blink renders HTML/CSS while V8 executes JavaScript.
Microsoft Edge	Blink	Modern Edge is Chromium-based and uses Blink; JavaScript uses V8.
Mozilla Firefox	Gecko	Uses Mozilla's Gecko engine; JavaScript is handled by SpiderMonkey.
Apple Safari	WebKit	Uses Apple's WebKit engine; JavaScript is handled by JavaScriptCore.
Opera	Blink	Modern Opera is Chromium-based and uses Blink; JavaScript uses V8.

Rendering-engine differences: A rendering engine parses HTML and CSS and turns them into the visual page. Different engines can vary in standards support, CSS behaviour, layout implementation, performance, security architecture, and adoption timing for new web-platform features. Browsers using the same engine generally have greater rendering compatibility, while browsers using different engines may show compatibility differences.

10. Basic Web Architecture Flow



Flow: The client (browser) sends a request to the server. The server runs application logic and may call APIs or query a database. The database stores and retrieves persistent data. APIs provide defined interfaces for communication between applications or services. The server then sends a response back to the client, which renders the result for the user.

Conclusion

Web development combines client-side technologies, server-side programming, databases, APIs, and development tools. Understanding the client-server model and the responsibilities of different project roles provides a foundation for building, testing, and maintaining modern web applications.